COMP1860 Computer Systems and Architecture

S1.W7.L1: Adders and the ALU

Leandro Soares Indrusiak

Last time

- The fetch–execute cycle; tracing programs on a simple accumulator machine (MARIE)
- Von Neumann vs Harvard; our Hack computer
- Instruction set architectures: stack, accumulator, register; addressing modes; RISC vs CISC
- Today: the arithmetic hardware that executes those instructions

Objectives

1. Build adders from logic gates: half adder, full adder, 16-bit ripple-carry adder
2. Explain why ripple-carry adders are slow, and how carry-lookahead speeds them up
3. Subtract in two's complement, and detect overflow
4. Understand the requirements of the ALU, and design a simple ALU for our Hack computer
5. Implement the chips required for the ALU in HDL

S1.W6.L2: PART I Arithmetic in gates

Binary addition (Recap)

x_n	y_n	Z_n carry to z_{n+1}
0	0	0 carry 0
0	1	1 carry 0
1	0	1 carry 0
1	1	0 carry 1

Binary addition example

	2^4	2^3	2^2	2^1	2^0
x	**0**	**0**	**1**	**1**	**1**
y	**1**	**0**	**1**	**1**	**0**
x+y					
Carry					

x_n	y_n	Z_n carry to z_{n+1}
0	0	0 carry 0
0	1	1 carry 0
1	0	1 carry 0
1	1	0 carry 1

Adders in logic

- Adding two bit's together can be represented by a truth table →
- Adding two bit's together can be computed by logic gates

x + y 低位进位 x+y的和 x+y的进位

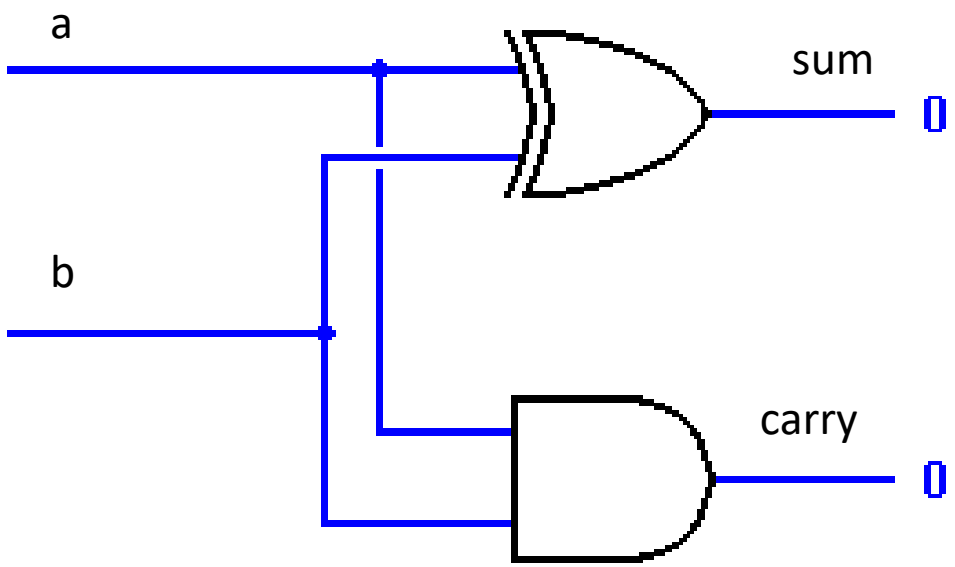
x	y	carry in	sum	carry
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1

Half-adder（半加器：不考虑低位进位）

- Adding two bit's together can be represented by a truth table
- Adding two bit's together can be computed by logic gates
- Haslf-adder has 2 input： x and y

x	y	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Half Adder Circuit




```
HalfAdder (a= ,b= ,sum= , carry= );
```

根据真值表　　xor gate　and gate

x	y	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

```
CHIP HalfAdder {
    IN a, b;    // 1-bit inputs
    OUT sum,    // Right bit of a + b
        carry;  // Left bit of a + b

    PARTS:
        Xor(a= a, b=b , out=sum );
        And(a=a , b=b , out=carry );
}
```

Full-Adder （全加器：考虑低位进位）

- Adding 3 bits together can be seen as adding 2 bits together followed by adding the 3rd bit to the result
- Adding 3 bits together can be achieved using two half-adders
- Full-adder has 3 input： x and y and carry in

What about adding more than single bits?

x + y 低位进位 x+y的和 x+y的进位

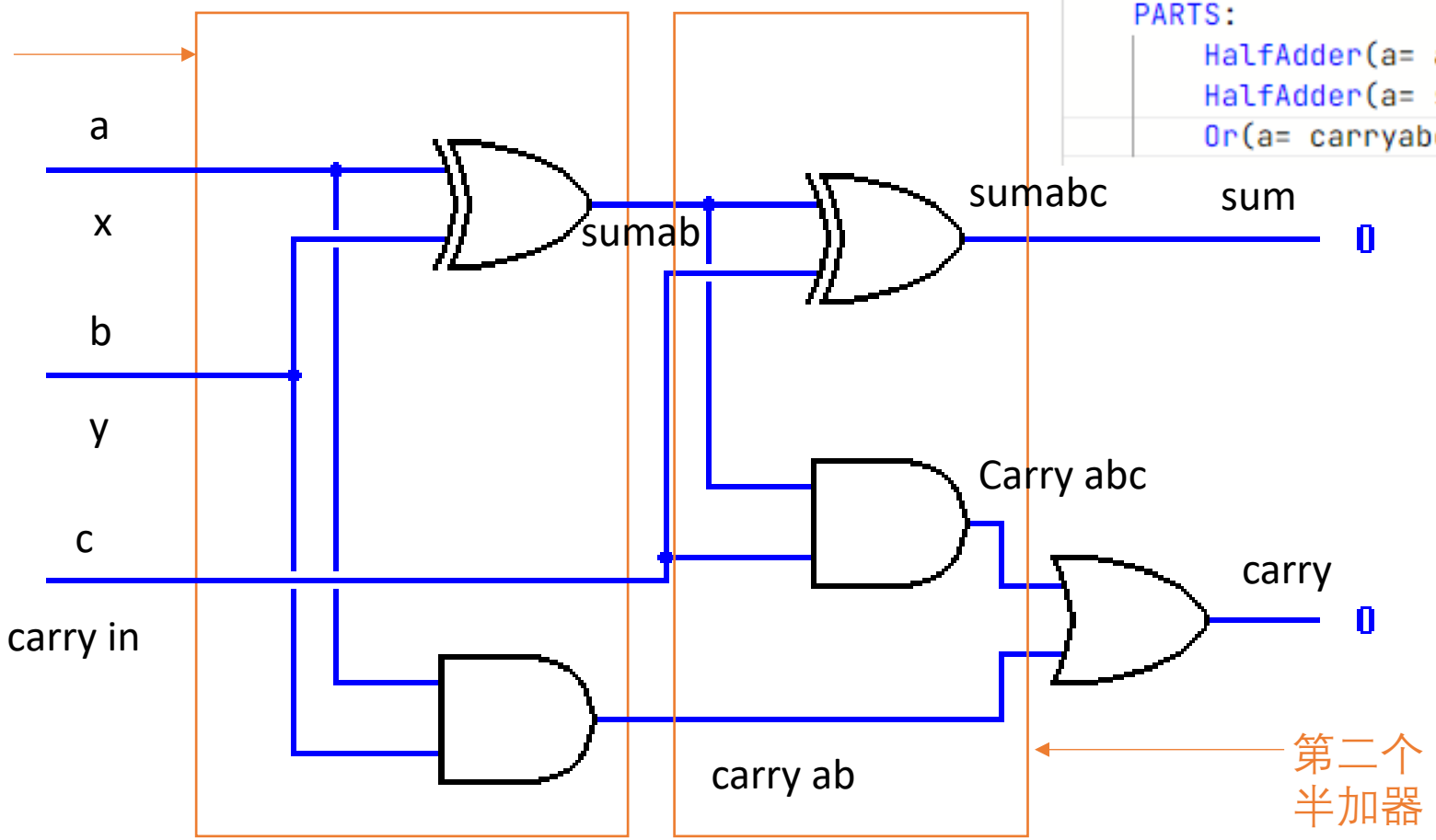
x	y	carry in	sum	carry
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1

Full Adder(两个半加器组成)

```
FullAdder (a= ,b= ,c= ,sum= ,carry= );
```

```
CHIP FullAdder {
    IN a, b, c;  // 1-bit inputs
    OUT sum,     // Right bit of a + b + c
        carry;   // Left bit of a + b + c

    PARTS:
        HalfAdder(a= a, b= b, sum=sumab , carry= carryab);
        HalfAdder(a= sumab, b= c, sum=sum , carry= carryabc);
        Or(a= carryabc, b=carryab , out= carry);
```



连接方法:

1. 第一个半加器的a，b接 x ， y
2. 第二个半加器的a，b接 第一个半加器的输出sum ab、低位进位carry in
3. 第二个半加器的carry = or (第一个半加器的输出carry b ,第二个半加器的输出carry abc)

16-bit Adder

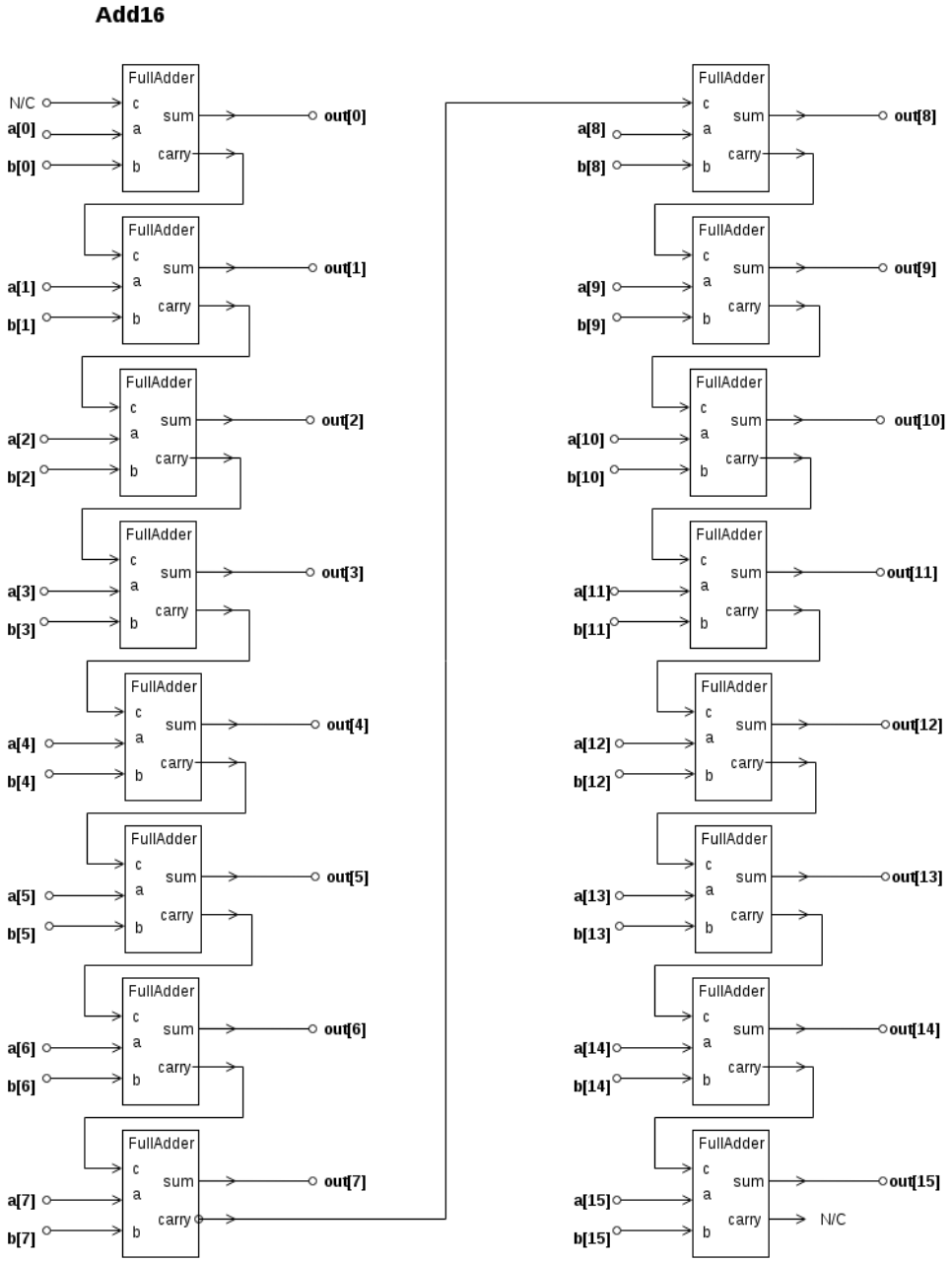
- By cascading full-adders into each other we can achieve a multi-bit adder
- No carry in for the first bit
- Carry out of the last bit is “lost”

通过将全加法器级联，我们可以实现多位加法器

最低位没有低位进位

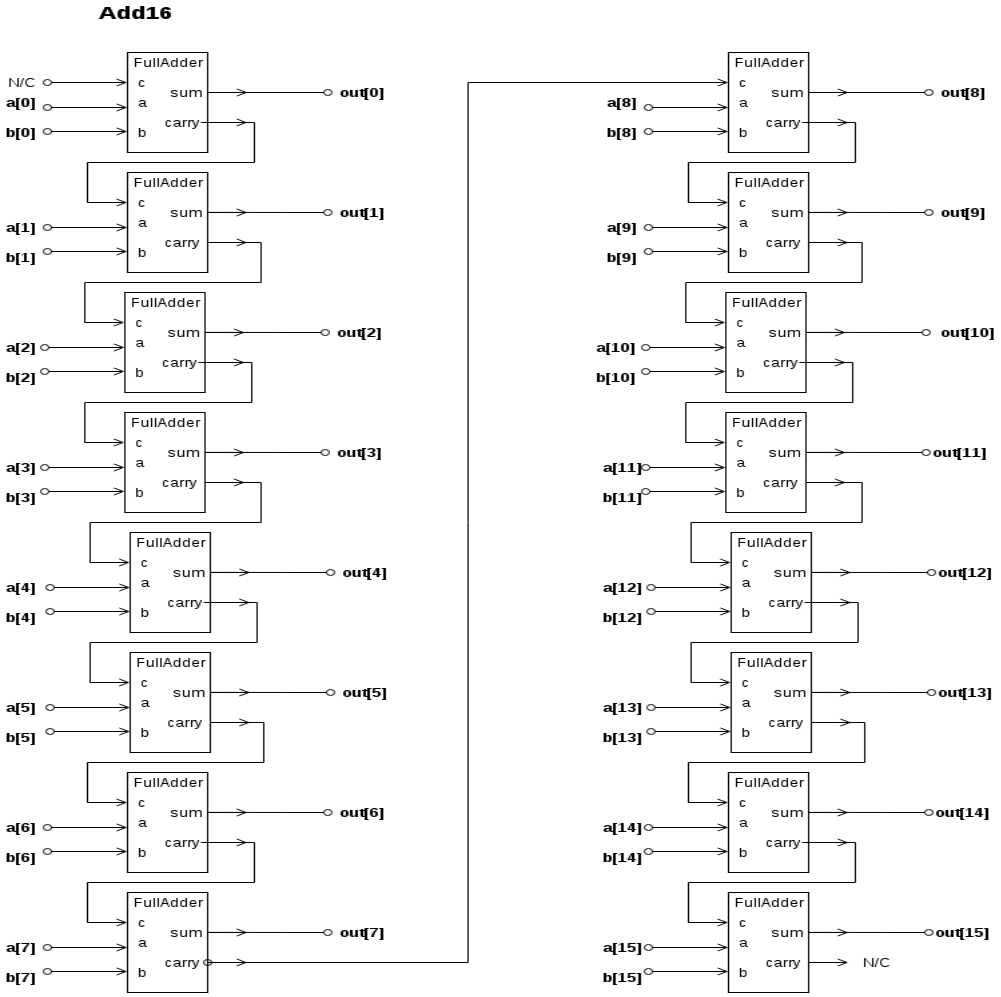
最高位的向上进位被丢弃

This format of multi-bit add is called a *ripple adder*(加法像涟漪ripple一样扩散)



Your turn: Can you build a 16-bit Adder

```
Add16 (a= ,b= ,out= );
```




```
CHIP Add16 {
    IN x[16], y[16];
    OUT out[16];

    PARTS:
    FullAdder(a=x[0] , b=y[0] , c=false , sum= out[0], carry=c0 );
    FullAdder(a=x[1] , b=y[1] , c=c0 , sum= out[1], carry=c1 );
    FullAdder(a=x[2] , b=y[2] , c=c1 , sum= out[2], carry=c2 );
    FullAdder(a=x[3] , b=y[3] , c=c2 , sum= out[3], carry=c3 );

    FullAdder(a=x[4] , b=y[4] , c=c3 , sum= out[4], carry=c4 );
    FullAdder(a=x[5] , b=y[5] , c=c4 , sum= out[5], carry=c5 );
    FullAdder(a=x[6] , b=y[6] , c=c5 , sum= out[6], carry=c6 );
    FullAdder(a=x[7] , b=y[7] , c=c6 , sum= out[7], carry=c7 );

    FullAdder(a=x[8] , b=y[8] , c=c7 , sum= out[8], carry=c8 );
    FullAdder(a=x[9] , b=y[9] , c=c8 , sum= out[9], carry=c9 );
    FullAdder(a=x[10], b=y[10], c=c9 , sum= out[10], carry=c10);
    FullAdder(a=x[11], b=y[11], c=c10, sum= out[11], carry=c11);

    FullAdder(a=x[12], b=y[12], c=c11, sum= out[12], carry=c12 );
    FullAdder(a=x[13], b=y[13], c=c12, sum= out[13], carry=c13 );
    FullAdder(a=x[14], b=y[14], c=c13, sum= out[14], carry=c14);
    FullAdder(a=x[15], b=y[15], c=c14, sum= out[15], carry=c15);
}
```

Your turn:

Build a chip that increments a value by one.

```
Inc16 (in= ,out= );
```

```
CHIP Inc16 {
    IN in[16];
    OUT out[16];

    PARTS:
     Add16(a= in, b[0]=true,b[1..15]=false , out= out);
}
```

How slow is a ripple-carry adder?(涟漪进位加法器有多慢？)

Every gate takes time to settle its output: one gate delay每个门都要时间稳定其输出：一个门延迟

In our full adder, the carry goes through 2 gates (AND, then OR) from carry-in to carry-out

Bit i cannot produce its final sum until the carry from bit i−1 has arrived

16-bit ripple-carry adder: the carry passes through 16 full adders

about 2 × 16 = 32 gate delays before the last bit is correct

Delay grows linearly with the number of bits: a 64-bit ripple-carry adder is 4 times slower

Can we compute every carry without waiting for the previous one?

在我们的全加器中，进位从进位到进位经过2个门（AND，然后OR），

在第i-1位的进位到达之前，第i位无法产生其最终总和

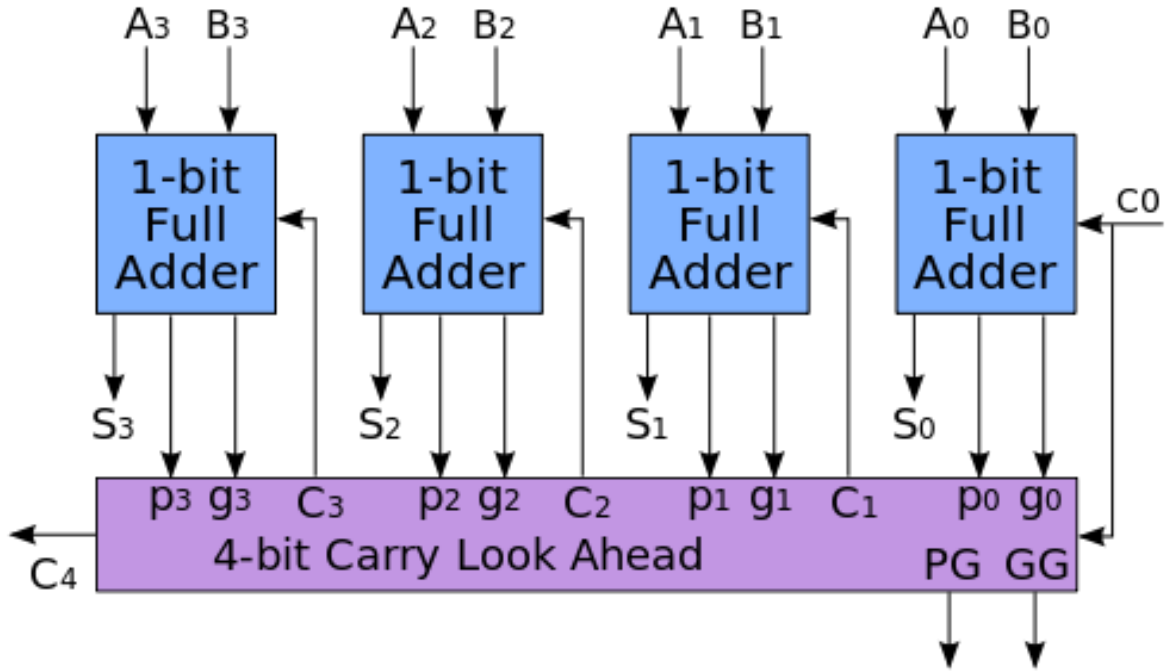
16位波纹进位加法器：进位通过16个全加法器。在最后一位正确之前，大约有2×16=32个门延迟

延迟随比特数线性增长：64位波纹进位加法器的速度慢4倍

我们能在不等待前一个进位的情况下计算每个进位吗？

Carry-Look ahead adder（进位前瞻加法器）

- Some problems with ripple-adders can be overcome by carry-look ahead add
- No more long propagation times
- At the expense of simplicity



- 没有更长的传播时间
- 牺牲了简洁性

Carry-Look ahead adder

We define two variables as **'carry generate'** G_i and **'carry propagate'** P_i then,

将两个变量定义为"进位-生成" G_i和"进位-传播" P_i"那么,

$$P_i = A_i \oplus B_i$$
$$G_i = A_i B_i$$

The sum output and carry output can be expressed in terms of carry generate G_i and carry propagate P_i as

sum输出 = G_i 进位输出= P_i

$$S_i = P_i \oplus C_i$$
$$C_{i+1} = G_i + P_i C_i$$

where G_i produces the carry when both A_i , B_i are 1 regardless of the input carry. P_i is associated with the propagation of carry from C_i to C_{i+1} .

A	B	C_i	C_o	Type of Carry
0	0	0	0	None
0	0	1	0	None
0	1	0	0	None
0	1	1	1	Propagate
1	0	0	0	None
1	0	1	1	Propagate
1	1	0	1	Generate
1	1	1	1	Generate/ Propagate

Carry-Look ahead adder

The carry output of each stage in a 4 stage carry look-ahead adder can be expressed as

$$C_1 = G_0 + P_0 C_{in}$$

$$C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_{in}$$

$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{in}$$

$$C_4 = G_3 + P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{in}$$

We can observe that C_4 does not have to wait for C_3 and C_2 to propagate but actually C_4 is propagated at the same time as C_3 and C_2 .

Since the Boolean expression for each carry output is the sum of products, these can be implemented with one level of AND gates followed by an OR gate.

https://www.geeksforgeeks.org/digital-logic/carry-look-ahead-adder/

Ripple-carry vs carry-lookahead

	Ripple-carry	**Carry-lookahead**
Carry into bit i	waits for the carry out of bit i−1	computed directly from the G, P and C_in signals
Delay (n bits)	grows with n: about 2n gate delays	constant: G, P (1) + carries as AND–OR (2) + sum XOR (1)
Hardware	n full adders	grows fast: C_n needs AND and OR gates with n+1 inputs
In practice	small, slow designs	4-bit lookahead blocks, combined in a hierarchy

Both are valid implementations of Add16: same interface (a, b, out), different speed and size.

Signed binary number(有符号二进制数，可 表示±数)

- 2’s complement（2的补码） can represent 2^n integers
- Maximum and minimum values are $2^{n-1}-1$ and -2^{n-1} (respectively)
- All positive numbers start with 0
- All negative numbers start with 1

Binary subtraction example（二进制减法）

	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
x	**0**	**0**	**0**	**1**	**0**	**1**	**1**	**0**
y	**0**	**0**	**0**	**0**	**0**	**1**	**1**	**1**
!y								
!y+1								
x+(!y+1)								
Carry								

x_n	y_n	Z_n carry to z_{n+1}
0	0	0 carry 0
0	1	1 carry 0
1	0	1 carry 0
1	1	0 carry 1

x - y = x + (-y) = x + (y的补码) = x + (y取反+1) = x +(!y+1)

Overflow in two's complement(补码的溢出问题)

With 16 bits we can represent −32768 … +32767; a result outside that range wraps around

4-bit examples (range −8 … +7):

```
0111 + 0001 = 1000      7 + 1 gives −8: overflow        正 + 正 = 负
1001 + 1110 = (1)0111   −7 + (−2) gives +7: overflow    负 + 负 = 正
1110 + 0011 = (1)0001   −2 + 3 gives +1: 产生进位，但无溢出  正+负 =正
```

Rule: overflow happens when both operands have the same sign and the result has the other sign

equivalently: carry into the top bit ≠ carry out of the top bit

Overflow判断规则： 当两个操作数具有**相同的符号**，但结果数的符号相反时，就会发生溢出

A discarded carry is not an error; overflow is

The PSW from last lecture has an overflow flag; the Hack ALU does not – software must avoid it

被丢弃的进位并非错误；溢出才是。

上一讲提到的程序状态字（PSW）带有溢出标志位；而Hack架构的ALU没有这一标志位——软件必须主动避免溢出。

Your turn: detect overflow

Build a 16-bit adder that also reports overflow:

```
Add16V (a= ,b= ,out= ,ovf= );
```

ovf = 1 when a and b have the same sign, but out has a different sign

Hint: the sign is bit 15; you can use your Add16 chip plus a few gates

Your turn: solution

```
CHIP Add16V {
    IN a[16], b[16];
    OUT out[16], ovf;

    PARTS:
    // 先执行补码加法，把结果最高位（符号位）放入内部变量s
    Add16(a=a, b=b, out=out, out[15]=s);

    // xor不同为1：把ab的最高位（符号位）进行xor，ab符号位不同时diffsign=1
    Xor(a=a[15], b=b[15], out=diffsign);   // operands differ in sign?

    // 对diffsign取反，得到同或效果，ab符号位相同时samesign=1
    Not(in=diffsign, out=samesign);

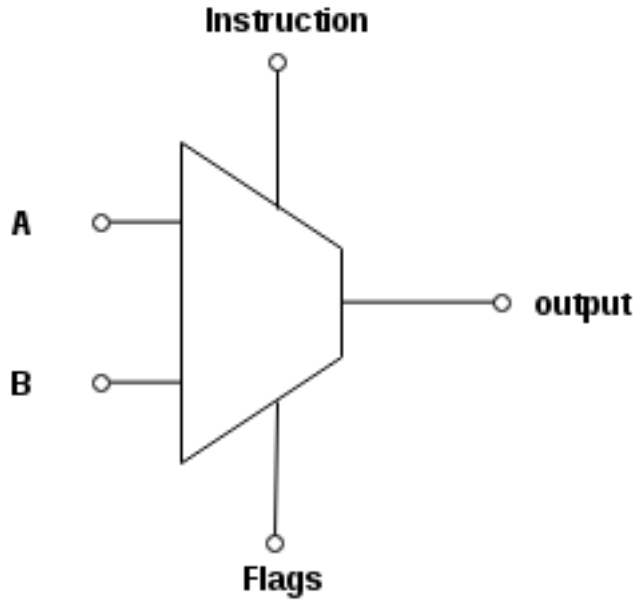
    // xor不同为1：flipped  = a的符号位 xor 结果符号位
    Xor(a=a[15], b=s, out=flipped);        // result sign differs from a?

    // ab符号位一样(samesign==1) 且 a符号位和结果符号位不一致(flipped == 1)，产生overflow
    And(a=samesign, b=flipped, out=ovf);
}
```

S1.W6.L2: PART II The Hack ALU

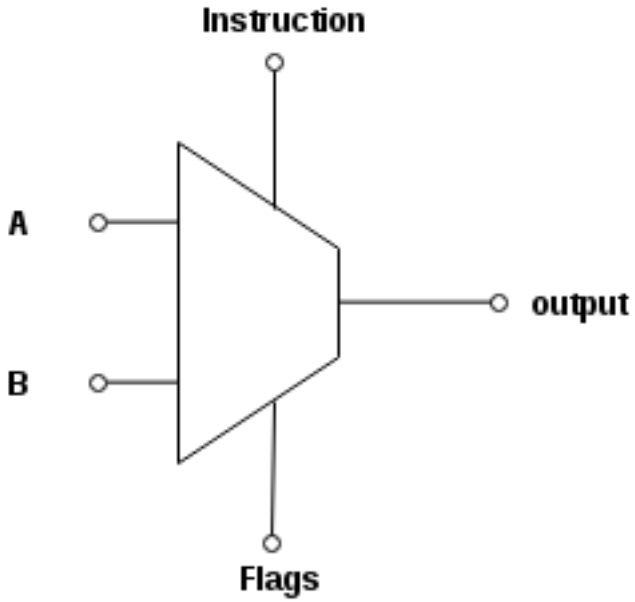
Arithmetic Logic Unit

- The Arithmetic Logic Unit (ALU) is the centerpiece of any modern day computer
- The ALU implements a set of logic commands
- The ALU allows for the selection of which command should be output



Arithmetic Logic Unit

- ALU's implement a fixed set of functions
- Each function can take at most two multi-bit arguments
- The ALU outputs the result of the boolean function and a set of flags



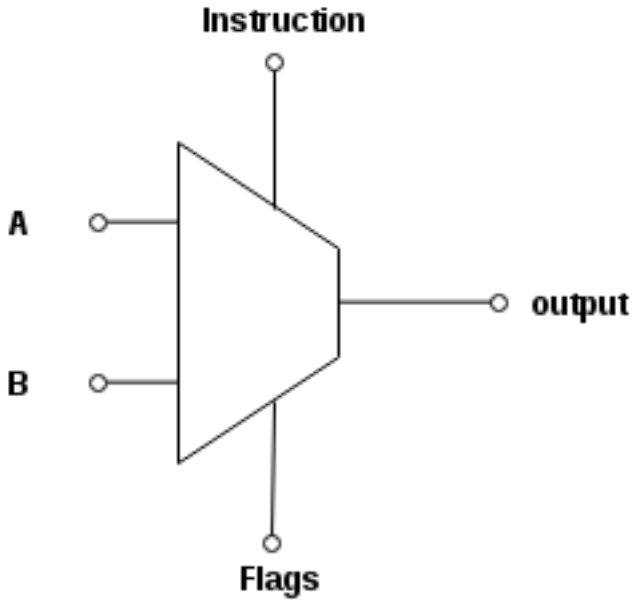
Design issue: Which functions should the ALU perform?

A hardware / software tradeoff: Any function not implemented by the ALU can be implemented later in system software

- Hardware implementations: Faster, and more expensive
- Software implementations: Slower, less expensive

Arithmetic Logic Unit

- The instruction is a set of control bits which select which function the ALU should output
  - And
  - Adder
  - Pre/post processing on inputs/outputs



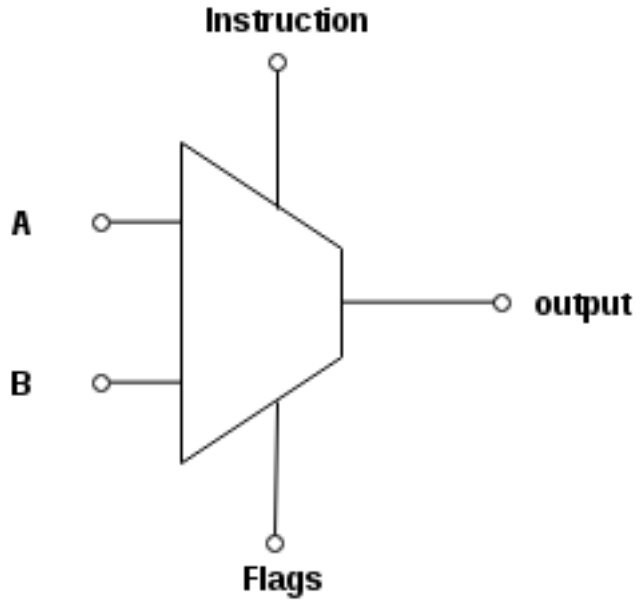
Arithmetic Logic Unit

Our specific ALU accepts

- Two 16-bit wide two's complement values
- A 6-bit wide instruction (control bits)

And outputs

- A 16-bit wide output
- A bit to indicate if the output is 0
- A bit to indicate if the output is less than 0

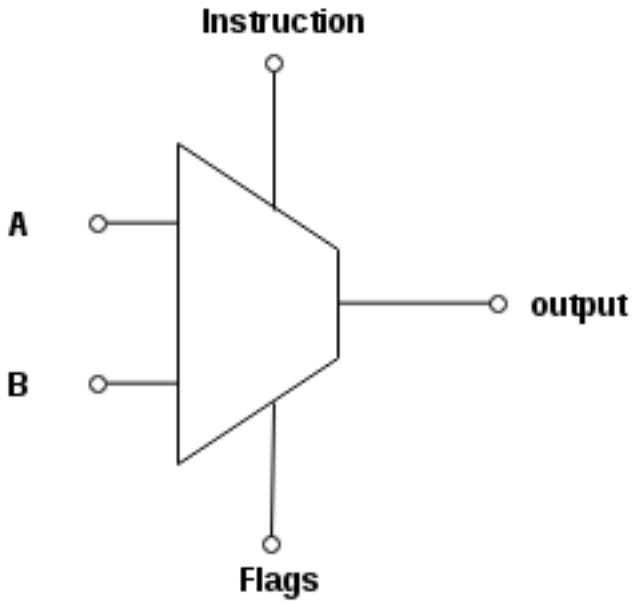


Arithmetic Logic Unit

- With a 6-bit wide instruction gives the ALU

  a possible 2^6 (=64) instructions

- We are only interested in 18 of them



out
0
1
-1
a
b
!a
!b
-a
-b
a+1
b+1
a-1
b-1
a+b
a-b
b-a
a&b
a|b

Control bits

- The ALU has the choice between two basic functions logical **And** and **Addition**
- All operations are multi-bit operations
- All of the required components have been covered previously

Bit	Description
za	Zero input **A**
na	Negate input **A**
zb	Zero input **B**
nb	Negate input **B**
f	Select between & and +
no	Negate output

Control bits

- Each of the 18 configurations of the control bits does something of interest
- Let us consider the control bits set to 001110
  - The ALU is instructed to compute a-1
  - Input **a** is neither zero'ed nor negated
  - Input **b** is zeroed and then negated (2's complement -1_{10})
  - f-bit set to **Addition**
  - Output is not negated

za	na	zb	nb	f	no	output
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	a
1	1	0	0	0	0	b
0	0	1	1	0	1	$\neg a$
1	1	0	0	0	1	$\neg b$
0	0	1	1	1	1	$-a$
1	1	0	0	1	1	$-b$
0	1	1	1	1	1	$a+1$
1	1	0	1	1	1	$b+1$
0	0	1	1	1	0	$a-1$
1	1	0	0	1	0	$b-1$
0	0	0	0	1	0	$a+b$
0	1	0	0	1	1	$a-b$
0	0	0	1	1	1	$b-a$
0	0	0	0	0	0	$a\&b$
0	1	0	1	0	1	$a\|b$

	za	na	zb	nb	f	no	output
	`if za then a=0`	`if na then a=!a`	`if zb then b=0`	`if nb then b=!b`	`if f then out=a+b else out=a&b`	`if no then output=!out`	`output(a,b)=`
	0	0	1	1	1	0	a-1
a							
b							

	za	na	zb	nb	f	no	output
	0	0	1	1	1	1	-a
a							
b							

	za	na	zb	nb	f	no	output
	`if za then a=0`	`if na then a=!a`	`if zb then b=0`	`if nb then b=!b`	`if f then out=a+b else out=a&b`	`if no then output=!out`	`output(a,b)=`
	0	0	1	1	1	0	a-1
a	a	a	a	a	a-1	a-1	a-1
b	b	b	0	-1 (11111111)			

	za	na	zb	nb	f	no	output
	0	0	1	1	1	1	-a
a							
b							

	za	na	zb	nb	f	no	output
	`if za then a=0`	`if na then a=!a`	`if zb then b=0`	`if nb then b=!b`	`if f then out=a+b else out=a&b`	`if no then output=!out`	`output(a,b)=`
	0	0	1	1	1	0	a-1
a	a	a	a	a	a-1	a-1	a-1
b	b	b	0	-1 (11111111)			

	za	na	zb	nb	f	no	output
	0	0	1	1	1	1	-a
a	a	a	a	a	a-1	!(a-1)	-a
b	b	b	0	-1			

Your turn: compute !a

	za	na	zb	nb	f	no	output
	`if za then a=0`	`if na then a=!a`	`if zb then b=0`	`if nb then b=!b`	`if f then out=a+b else out=a&b`	`if no then output=!out`	`output(a,b)=`
a=1100							
b=1011							

Your turn: compute !a

	za	na	zb	nb	f	no	output
	`if za then a=0`	`if na then a=!a`	`if zb then b=0`	`if nb then b=!b`	`if f then out=a+b else out=a&b`	`if no then output=!out`	`output(a,b)=`
	0	0	1	1	0	1	!a
a=1100	1100	1100	1100	1100	1100	0011	!a
b=1011	1011	1011	0000	1111			

Your turn: compute b-a

	za	na	zb	nb	f	no	output
	`if za then a=0`	`if na then a=!a`	`if zb then b=0`	`if nb then b=!b`	`if f then out=a+b else out=a&b`	`if no then output=!out`	`output(a,b)=`
a=0010(2)							
b=0111(7)							

Your turn: compute b-a

	za	na	zb	nb	f	no	output
	`if za then a=0`	`if na then a=!a`	`if zb then b=0`	`if nb then b=!b`	`if f then out=a+b else out=a&b`	`if no then output=!out`	`output(a,b)=`
	0	0	0	1	1	1	b-a
a=0010(2)	0010	0010	0010	0010	1010	0101	5
b=0111(7)	0111	0111	0111	1000			

Why does it work?

In two's complement, negation is: invert all bits, then add 1

```
−x = !x + 1
```

Rearranging:

```
!x = −x − 1
```

Check with 4 bits: x = 0101 (5)

`!x = 1010` which is −6 = −5 − 1, as expected

So each negate bit (na, nb, no) is arithmetic negation minus one

Also: all ones = !0 = −1

Deriving the rows from the identity

`a−1  (001110)`: b = !0 = −1; out = a + (−1) = a − 1

`−a  (001111)`: out = !(a − 1) = −(a − 1) − 1 = −a

`a+1  (011111)`: a → !a = −a − 1; b → −1; sum = −a − 2

out = !(−a − 2) = −(−a − 2) − 1 = a + 1

`a−b  (010011)`: a → !a = −a − 1; sum = −a − 1 + b

out = !(−a − 1 + b) = −(−a − 1 + b) − 1 = a − b

Each row is just za/na, zb/nb, f, no applied in order

Your turn: derive these rows

`b-a` `(000111)`

`b+1` `(110111)`

`a|b` `(010101)` hint: De Morgan's law

Your turn: solution

`b−a`: b → !b = −b − 1; sum = a − b − 1

out = !(a − b − 1) = −(a − b − 1) − 1 = b − a

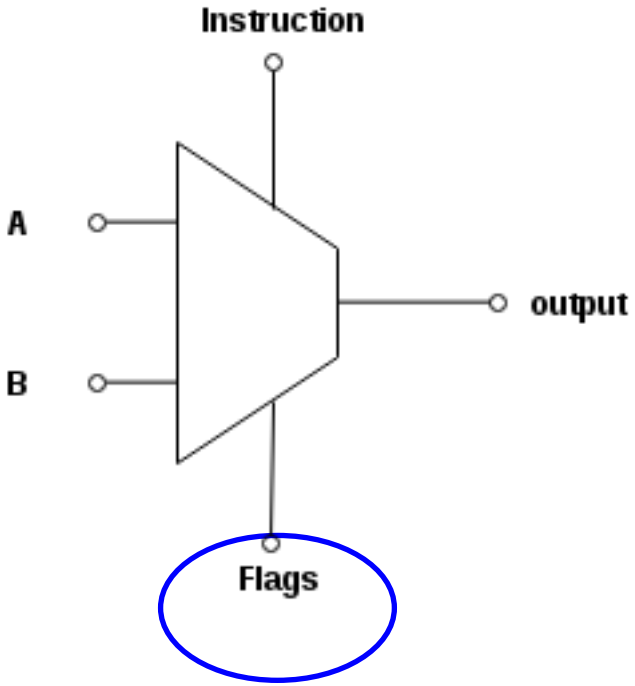
`b+1`: a → !0 = −1; b → !b = −b − 1; sum = −b − 2

out = !(−b − 2) = b + 2 − 1 = b + 1

`a|b`: a → !a; b → !b; f = & gives !a & !b

out = !(!a & !b) = a | b (De Morgan)

The Output Flags



if (`out == 0`) then `zr = 1`, else `zr = 0`

if (`out < 0`) then `ng = 1`, else `ng = 0`

The `zr` and `ng` output bits will come into play when we'll build the complete CPU architecture, later in the course.

ALU

- The ALU we implement is quite basic
- No multiplication/division
- No floating point numbers
- No exponentiation (required for some encryptions)
- Modern ALU’s support many more operations

Implementation

Notice that the ALU is composed of

- A logic circuit that can negate or zero an input (times 2)
- Logic circuits for **Addition** and **And.**
- A logic circuit to select between them
- A logic circuit to negate the output
- A logic circuit to handle flags

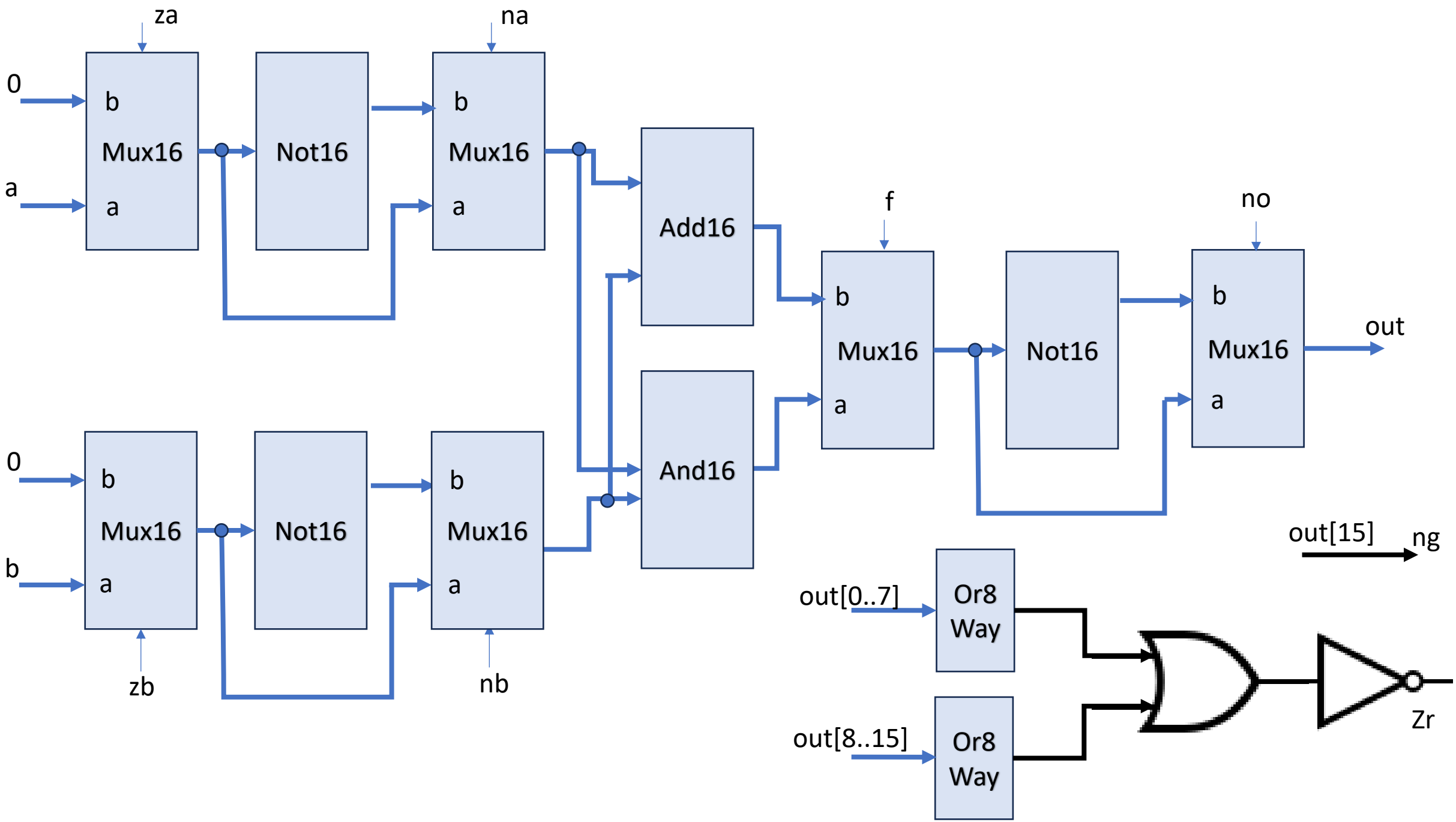
The Hack ALU operation

pre-setting the a input		pre-setting the b input		selecting between computing + or &	post-setting the output	Resulting ALU output
za	**na**	**zb**	**nb**	**f**	**no**	**out**
if za then a=0	if na then a=!a	if zb then b=0	if nb then b=!b	if f then out=a+b else out=a&b	if no then out=!out	out(a,b)=
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	a
1	1	0	0	0	0	b
0	0	1	1	0	1	!a
1	1	0	0	0	1	!b
0	0	1	1	1	1	-a
1	1	0	0	1	1	-b
0	1	1	1	1	1	a+1
1	1	0	1	1	1	b+1
0	0	1	1	1	0	a-1
1	1	0	0	1	0	b-1
0	0	0	0	1	0	a+b
0	1	0	0	1	1	a-b
0	0	0	1	1	1	b-a
0	0	0	0	0	0	a&b
0	1	0	1	0	1	a|b

Your turn to build the ALU

```
ALU (a= ,b= ,za= ,na= ,zb= ,nb= ,f= ,no= ,out= ,zr= ,ng= );
```

First sketch out the circuit using blocks to represent the parts of the chip and then have a go at implementing it.


za
na
0
b
Mux16
a
a
Not16
b
Mux16
a
Add16
f
b
Mux16
a
Not16
no
b
Mux16
a
out
0
b
Mux16
a
b
Not16
b
Mux16
a
And16
zb
nb
out[15]
ng
out[0..7]
Or8
Way
out[8..15]
Or8
Way
Zr

Driving the ALU from an instruction

Bits	**Meaning in a Hack computation instruction**
15–13	1 1 1: marks this as a computation instruction
12	selects the ALU's second input b: the A register or the memory word M[A]
11–6	the six ALU control bits: za na zb nb f no
5–3	where to store the result: A, D and/or M[A]
2–0	jump condition, decided using the ALU's zr and ng flags

The ALU's first input a is always the D register
Decoding is mostly wiring: bits 11–6 go straight to the ALU
The assembly language for these instructions comes next semester

Summary

- Built the half adder, full adder and 16-bit ripple-carry adder in HDL
- Saw why the ripple carry is slow and how carry-lookahead removes the chain
- Subtraction and overflow in two's complement
- Designed the Hack ALU: six control bits, why they work, and the zr/ng flags
- Saw how a computation instruction drives the ALU control bits